

RLHF平替算法DPO篇

来自：AiGC面试宝典

Just do it!

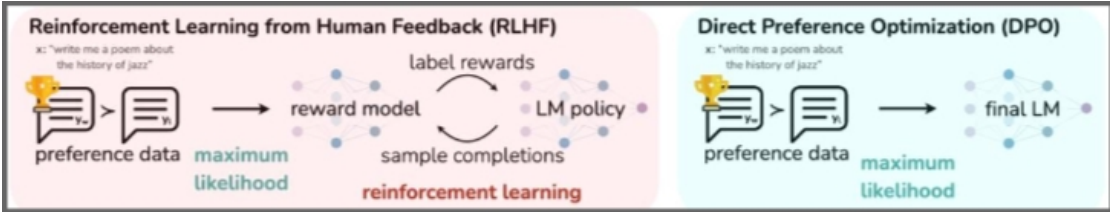
2024年05月06日 22:02



RLHF平替算法DPO篇

- 一、 DPO vs RLHF?
- 二、 介绍一下 DPO的损失函数?
- 三、 DPO 微调流程 ?
- 四、 说一下 DPO 是如何简化 RLHF 的?
- 五、 DPO的第0步loss是固定的么? 如果固定的话, 值是多少?
- 六、 DPO是一个on-policy还是off-policy的算法, 以及这样的算法有什么优劣?
- 七、 DPO公式是由PPO的objective公式推导过来的, 为什么DPO是off-policy算法, 而PPO是on-policy算法, 到底哪一步推导出了问题?
- 八、 DPO为什么会在学习过程中training positive的概率和training negative的概率都同时下降?
- 九、 在什么情况下DPO exactly 数学上等同于 PPO?
- 十、 DPO的变体有哪些, 主要解决DPO的什么问题?
- 十一、 DPO训练后的模型为什么会输出越来越长?
- 代码解释
 - 1. DPO损失函数 代码实现?
 - 2. DPO 批次训练过程 代码实现?
 - 3. LM的交叉熵计算 代码实现?

一、 DPO vs RLHF?



上图左边是 RLHF 算法，右边为 DPO 算法，两图的差异对比即可体现出 DPO 的改进之处。

1. RLHF 算法:包含奖励模型(reward model)和策略模型(policy model, 也称为演员模型, actor model), 基于偏好数据以及强化学习不断迭代优化策略模型的过程。
2. DPO 算法:不包含奖励模型和强化学习过程, 直接通过偏好数据进行微调, 将强化学习过程直接转换为 SFT 过程, 因此整个训练过程简单、高效, 主要的改进之处体现在于损失函数。

ps:

1. 偏好数据, 可以表示为三元组(提示语 prompt, 良好回答 chosen, 一般回答 rejected)。论文中的 chosen 表示为下标 w(即 win), rejected 表示为下标 l(即 lose)
2. RLHF 常使用 PPO 作为基础算法, 整体流程包含了 4 个模型, 且通常训练过程中需要针对训练的 actor model 进行采样, 因此训练起来, 稳定性、效率、效果不易控制。
 - a. actor model/policy model: 待训练的模型, 通常是 SFT 训练后的模型作为初始化
 - reference model: 参考模型, 也是经 SFT 训练后的模型进行初始化, 且通常与 actor model 是同一个模型, 且模型冻结, 不参与训练, 其作用是在强化学习过程中, 保障 actor model 与 reference model 的分布差异不宜过大。
 - reward model: 奖励模型, 用于提供每个状态或状态动作对的即时奖励信号。
 - Critic model: 作用是估计状态或状态动作对的长期价值, 也称为状态值函数或动作值函数。
3. DPO 算法仅包含 RLHF 中的两个模型, 即演员模型(actor model)以及参考(reference model), 且训练过程中不需要进行数据采样。

二、介绍一下 DPO 的损失函数?

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Diagram labels for the equation components:

- π_{θ} : Policy to optimize
- π_{ref} : Reference policy (used to control behavior of LLMs)
- $\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}$: Aggregation over preference data
- $\log \sigma$: Logistic function
- $\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)}$: Shift in preferred completion
- $\beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)}$: Shift in dispreferred completion

DPO 损失函数

如何将 RLHF 的 Reward model 过程简化为上式，作者花了大量篇幅进行了推导，感兴趣的读者可以参考附件 DPO 的论文。

DPO 算法的目的是最大化奖励模型(此处的奖励模型即为训练的策略),使得奖励模型对 chosen 和 rejected 数据的差值最大，进而学到人类偏好。

上式的后半部分通过对数函数运算规则，可以进行如下转化。

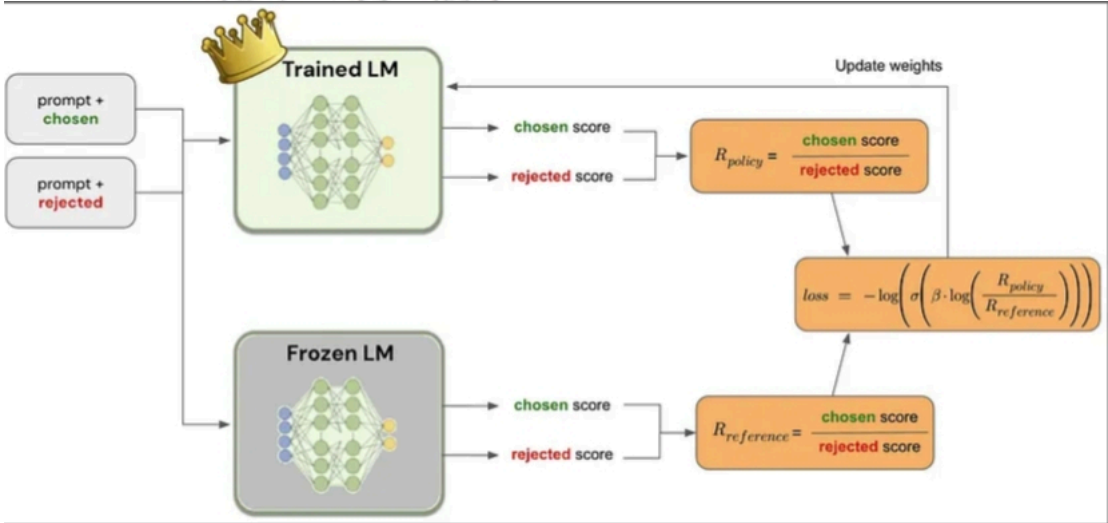
$$\begin{aligned} & \left(\beta \log \left(\frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} \right) - \beta \log \left(\frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right) \\ &= \beta \left(\pi_{\theta}(y_w|x) - \pi_{ref}(y_w|x) \right) - \beta \left(\pi_{\theta}(y_l|x) - \pi_{ref}(y_l|x) \right) \\ &= \beta \left(\pi_{\theta}(y_w|x) - \pi_{\theta}(y_l|x) \right) - \beta \left(\pi_{ref}(y_w|x) - \pi_{ref}(y_l|x) \right) \\ &= \beta \left(\pi_{\theta}(y_w|x) - \pi_{\theta}(y_l|x) - (\pi_{ref}(y_w|x) - \pi_{ref}(y_l|x)) \right) \end{aligned}$$

Loss 公式转换

转化后的公式和源代码中的计算函数中的公式是一致的。

其中左半部分是训练的 policy 模型选择 chosen 优先于 rejected，右半部分是冻结的 reference 模型选择 chosen 优先于 rejected，二者的差值可类似于 KL 散度，保障 actor 模型的分布与 reference 模型的分布不会有较大的差异。

三、DPO 微调流程？



DPO 微调流程

上图展示了 DPO 微调的大致流程，其中 Trained LM 即为策略模型，Frozen LM 即为参考模型，二者均是先进行 SFT 微调得到的模型进行初始化，其中 Trained LM 需要进行训练，Frozen LM 不参与训练。

两个模型分别针对 chosen 和 rejected 进行预测获取对应的得分，再通过 DPO 的损失函数进行损失计算，进而不断的迭代优化。

四、说一下 DPO 是如何简化 RLHF 的？

- RLHF 是如何训练？

RLHF 一般会分 2 步：

1. 第一步是训练 reward model。训练数据是同一个 prompt 的 2 个回答，让人或 GPT4 标注哪个回答更好，reward model 会去优化如下的 loss：

$$\max_{r_{\phi}} \left\{ \mathbb{E}_{(x, y_{\text{win}}, y_{\text{lose}}) \sim \mathcal{D}} [\log \sigma(r_{\phi}(x, y_{\text{win}}) - r_{\phi}(x, y_{\text{lose}}))] \right\}$$

其中 r 就是 reward model 用来给回答打分。 $\mathcal{D}$ 是训练数据集， x 是 prompt， y_{win} 和 y_{loss} 分别是好的回答和不好的回答。也就是说，要尽可能让好的回答的得分比不好的回答高，拉大他们之间的差别。

2. 第二步是用 RL 算法来提升模型的得分。使用的 loss 是：

$$\max_{\pi_{\theta}} \left\{ \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta}(y|x) || \pi_{\text{ref}}(y|x)] \right\}$$

其中 π_θ 是我们在训练的 LLM, π_{ref} 是训练的初始值。这个 loss 意思是希望 LLM 输出的回答的评分能尽可能高, 同时 π_θ 不要偏离 π_{ref} 太多, 保证它还能正常做回答, 不要训成一个评分很高但是回答乱码的东西。

• DPO 优化策略?

DPO 的作者们意识到, 后面的这个式子是有显式解的。因为:

$$\begin{aligned} \max_{\pi_\theta} \{ & \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)] \} \\ &= \max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}] \\ &= \min_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r_\phi(x, y)] \\ &= \min_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x) e^{r_\phi(x, y)/\beta}}] \end{aligned}$$

如果我们归一化一下分母, 即取

$$Z(x) = \sum_y \pi_{\text{ref}}(y|x) e^{r_\phi(x, y)/\beta}$$

也就可以构造出一个新的概率分布:

$$\pi^*(y|x) = \pi_{\text{ref}}(y|x) e^{r_\phi(x, y)/\beta} / Z(x)$$

那么上式变成了:

$$\begin{aligned} \min_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x) e^{r_\phi(x, y)/\beta}}] \\ &= \min_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [\log \frac{\pi_\theta(y|x)}{\pi^*(y|x)} - \log Z(x)] \\ &= \min_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [\log \frac{\pi_\theta(y|x)}{\pi^*(y|x)}] \\ &= \min_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{D}_{\text{KL}}(\pi_\theta(y|x) || \pi^*(y|x)) \end{aligned}$$

由于 KL 散度在 2 个分布相等时取最小值，我们得到了这样的结论：RLHF 训练希望得到的最优的概率分布就是 π^* 。

另一个角度来说，由 π^* 的公式，我们相当于得到了 r 和 π^* 的关系，那么是否我们可以把训练 r 转化成直接去训练 π^* 呢？

简单转换一下 π^* 的定义式，可以得到：

$$r_\phi(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

带入最上面优化 r 的 loss，也就有了：

$$\max_{\pi^*} \left\{ \mathbb{E}_{(x, y_{\text{win}}, y_{\text{lose}}) \sim \mathcal{D}} [\log \sigma(\beta \log \frac{\pi^*(y_{\text{win}}|x)}{\pi_{\text{ref}}(y_{\text{win}}|x)} - \beta \log \frac{\pi^*(y_{\text{lose}}|x)}{\pi_{\text{ref}}(y_{\text{lose}}|x)})] \right\}$$

或者说，我们可以直接用这个 loss 去求 π_θ ：

$$\max_{\pi_\theta} \left\{ \mathbb{E}_{(x, y_{\text{win}}, y_{\text{lose}}) \sim \mathcal{D}} [\log \sigma(\beta \log \frac{\pi_\theta(y_{\text{win}}|x)}{\pi_{\text{ref}}(y_{\text{win}}|x)} - \beta \log \frac{\pi_\theta(y_{\text{lose}}|x)}{\pi_{\text{ref}}(y_{\text{lose}}|x)})] \right\}$$

这就是 DPO 的 loss。DPO 通过以上的公式转换把 RLHF 无损地转化为了 SFT，在训练的时候不再需要同时跑 4 个模型（reward model, ref model, critic, actor），而是只用跑 actor 和 ref 2 个模型，甚至由于不再在线采数据，ref model 的输出可以预先存下来，训练的时候重复使用。

五、DPO的第0步 loss是固定的么？如果固定的话，值是多少？

是固定的，因为 DPO loss 为：

$$loss = -\log\text{sigmoid}(\beta \frac{p_{\theta}(y_w|x)}{p_{ref}(y_w|x)} - \beta \frac{p_{\theta}(y_l|x)}{p_{ref}(y_l|x)}),$$

其中 y_w 是 positive 的 y ，而 y_l 是 negative 的 y 。那么开始的时候由于优化的网络参数等于 reference 的网络参数，因此

$$p_{\theta}(y_w|x) = p_{ref}(y_w|x)$$

同理可得

$$p_{\theta}(y_l|x) = p_{ref}(y_l|x)$$

故

$$loss = -\log\text{sigmoid}(\beta - \beta)$$

这个数应该=0.693。

六、DPO是一个 on-policy 还是 off-policy 的算法，以及这样的算法有什么优劣？

DPO 是一个 off-policy 的算法，因为训练 DPO 的 pair 数据不一定来自 ref policy 或者 sft policy。优势是不需要对模型进行采样，然后标注，直接可以拿已有的数据集进行训练，这样的情况下包括采样的成本和标注的成本都可以节约。劣势是效果很难保证，尤其是你的模型本身能力和发布的 pair 数据不匹配的时候。相比而言，PPO 是一个 on-policy 的算法，整体效果会比 DPO 要好。

可以参考：

强化学习中 on-policy 与 off-policy 有什么区别？

<https://www.zhihu.com/question/57159315/answer/2226476385>

七、DPO 公式是由 PPO 的 objective 公式推导过来的，为什么 DPO 是 off-policy 算法，而 PPO 是 on-policy 算法，到底哪一步推导出了问题？

在 DPO 公式推导中，由目标公式：

$$\max_{\pi_{\theta}} E_{x \sim D, y \sim \pi_{\theta}} [r(x, y)] - \beta D_{KL}[\pi_{\theta}(y|x) || \pi_{ref}(y|x)]$$

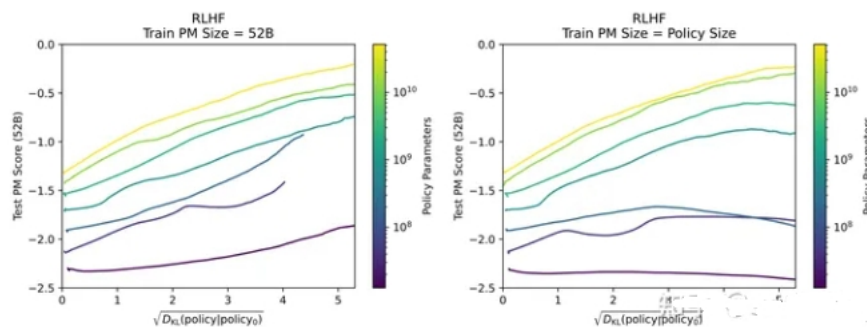
推导出 optimal policy

$$\pi_r(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) \exp(\frac{1}{\beta} r(x, y))$$

在公式中其实 π_{ref} 应该是随着模型更新而一直改变的，但是真正实现的时候一般使用 π_{sft} 代替。那么就导致了 DPO 从 on-policy 变成了 off-policy 的方法。DPO 面临着 RL 领域经典的 state distribution shift 的问题，从而效果会不如 PPO。除此之外由于 DPO 中 π_{ref} 和 π_{sft} 有 KL 散度的限制，所以 state distribution shift 的问题不会像传统 RL 中那么大，所以整体上还是 work 的。

补：经修正，相比于不加 KL 散度或者传统 bandit 算法算是分布差异小，

但整体分布差异仍然很大（约 25 左右），如图：



KL 散度的差异和 Test PM Score 变化图

八、DPO为什么会在学习过程中 training positive 的概率和 training negative 的概率都同时下降？

因为 DPO 的 loss 是 BT loss，是 maximize training set 中 positive 和 negative 的 gap。那从公式上它就无法保证 training positive 的概率是一直上升的。那继续探究它背后的原因，主要和采样的方式以及 DPO loss 组成相关，

首先还是把 DPO loss 列出来：

$$loss = -\text{logsigmoid}(\beta \frac{p_{\theta}(y_w|x)}{p_{ref}(y_w|x)} - \beta \frac{p_{\theta}(y_l|x)}{p_{ref}(y_l|x)})$$

- 采样方式，一般dpo的数据采样来自模型内部生成（也就是所谓的on-policy采样），那么两个 response 的 token level 的重合度比较高，那么两个 response 的编辑距离会比较低。
- 在同一模型中，我们有个假设当 response 的编辑距离 $D(y_w, y_l) < T$ ，我们就有 $|p(y_w|x) - p(y_l|x)| < S$ ，因此 $|\beta \frac{p_{\theta}(y_w|x)}{p_{ref}(y_w|x)} - \beta \frac{p_{\theta}(y_l|x)}{p_{ref}(y_l|x)}| < H$ ，由于 logloss 的性质，这个公式只能在 $p_{\theta}(y_w|x)$ 和 $p_{\theta}(y_l|x)$ 越来越小的时候 loss 持续变小，可能之前的推导有些不严谨，举个例子大家更形象一些：
 - $p_{ref}(y_w|x) = 0.4$ ， $p_{ref}(y_l|x) = 0.3$ ，假设 $S = 0.1$ ，
 - 那么如果正例概率升高 $p_{\theta}(y_w|x) = 0.5$ ，由于 $|p(y_w|x) - p(y_l|x)| < S$ ，所以 $p_{\theta}(y_l|x) > 0.4$ ，假设 $\beta = 0.1$ ，所以 loss 一定大于 0.697。
 - 如果 $p_{\theta}(y_w|x) = 0.21$ ，那么 $p_{\theta}(y_l|x) > 0.11$ ，那么 loss 一定大于 0.685。
 - 如果 $p_{\theta}(y_w|x) = 0.11$ ，那么 $p_{\theta}(y_l|x) > 0.01$ ，那么 loss 一定大于 0.681。

整个数学的过程可能不那么严谨，但也是给大家一个形象的视角来看这个问题。

附计算代码：

```
import numpy as np
x = 0.00001
x_ref = 0.4
y = 0.0
y_ref = 0.3
beta = 0.1
gap = beta * (x / x_ref - y / y_ref)
print(gap)
log_sigmoid_value_pos = -np.log(1 / (1 + np.exp(-gap)))
print(log_sigmoid_value_pos)
```

九、在什么情况下DPO exactly 数学上等同于 PPO？

- $p(y > y') = \delta(r(y) - r(y'))$ 。
- y' 通过蒙特卡洛采样。
- DPO loss 需要改成 IPO loss 形式，也就是

$$\max_{\pi} E_{x \sim \rho, y \sim \pi(\cdot|x), y' \sim \mu(\cdot|x)} [\phi(p^*(y > y'|x))] - \tau D_{KL}(\pi || \pi_{ref})$$
- $\phi(q) = \frac{q}{1-q}$ 。

参考文献IPO， A General Theoretical Paradigm to Understand Learning from Human Preferences， 细节证明可以看IPO。

十、DPO的变体有哪些，主要解决DPO的什么问题？

- RSO [1]: 由于DPO的蒙特卡洛采样很难达到，所以其实DPO几乎是off-policy的采样方式，RSO主要从DPO的采样方式来解决DPO的问题。
- Iterative DPO [2]: 同样由于DPO的蒙特卡洛采样很难达到，所以通过on-policy的方式采样来替代off-policy的采样。
- IPO [3]: 由于BT model的目标是最大化正负response的reward gap，但其实其中忽略了真实情况下我们组的pair可能会有噪音，那么无限去扩大reward gap其实是不准确的，也就是overfit了preference的pair数据，那么解决方案是需要限制这个gap的范围。
- DPOP [4]: 由于LLM model很难区分编辑距离较小的pair，那么当持续去区分这批case的时候，模型效果会崩塌，现象是正例子和负例子的概率都往下掉。那么DPOP用了一个新项来惩罚正例往下掉的pair，使得正例概率继续提升。

[1] Liu T, Zhao Y, Joshi R, et al. Statistical rejection sampling improves preference optimization[J]. arXiv preprint arXiv:2309.06657, 2023.

[2] Yuan W, Pang R Y, Cho K, et al. Self-rewarding language models[J]. arXiv preprint arXiv:2401.10020, 2024.

[3] Azar M G, Rowland M, Piot B, et al. A general theoretical paradigm to understand learning from human preferences[J]. arXiv preprint arXiv:2310.12036, 2023.

[4] Pal A, Karkhanis D, Dooley S, et al. Smaug: Fixing Failure Modes of Preference Optimisation with DPO-Positive[J]. arXiv preprint arXiv:2402.13228, 2024.

十一、DPO训练后的模型为什么会输出越来越长？

并不是一定会越来越长。如果你尝试用所有正例子的response都比负例子的短，那么也会输出越来越短。究其原因，是由于数据构造原因导致的DPO后训练后的模型输出越来越长。因为，在短的response中一句话结束后的概率会很大，但是在长的response中，“但是”，“而且”等细节描述词会接在一句话后，那么这些词语的概率会由DPO过程逐渐变大。

代码解释

1. DPO损失函数 代码实现？

```
def preference_loss(policy_chosen_logps: torch.FloatTensor,
                    policy_rejected_logps: torch.FloatTensor,
                    reference_chosen_logps: torch.FloatTensor,
                    reference_rejected_logps: torch.FloatTensor,
                    beta: float,
                    label_smoothing: float = 0.0,
                    ipo: bool = False,
                    reference_free: bool = False) -> Tuple[torch.FloatTensor, torch.FloatTensor,
torch.FloatTensor]:
    # policy_chosen_logps: 训练模型对于chosen经过log后logits
    # policy_rejected_logps: 训练模型对于rejected经过log后logits
    # reference_chosen_logps: 训练模型对于chosen经过log后logits
    # reference_rejected_logps: 训练模型对于rejected经过log后logits
    # beta: policy和reference的差异性控制参数

    # actor模型选择chosen优先于rejected
    pi_logratios = policy_chosen_logps - policy_rejected_logps

    # reference模型选择chosen优先于rejected
    ref_logratios = reference_chosen_logps - reference_rejected_logps

    if reference_free:
```

```

ref_logratios = 0

# 差值可类似于KL散度, 保障actor模型的分布与reference模型的分布不会有较大的差异
logits = pi_logratios - ref_logratios # also known as  $h_{\{\pi_{\theta}\}^{y_w, y_l}}$ 

if ipo:
    losses = (logits - 1/(2 * beta)) ** 2 # Eq. 17 of https://arxiv.org/pdf/2310.12036v2.pdf
else:
    # Eq. 3 https://ericmitchell.ai/cdpo.pdf; label_smoothing=0 gives original DPO (Eq. 7 of
    https://arxiv.org/pdf/2305.18290.pdf)
    # label_smoothing为0, 对应的DPO论文的算法
    losses = -F.logsigmoid(beta * logits) * (1 - label_smoothing) - F.logsigmoid(-beta * logits)
    * label_smoothing

# chosen和rejected的奖励
chosen_rewards = beta * (policy_chosen_logps - reference_chosen_logps).detach()
rejected_rewards = beta * (policy_rejected_logps - reference_rejected_logps).detach()

return losses, chosen_rewards, rejected_rewards

```

2. DPO 批次训练过程 代码实现?

```

def get_batch_metrics(self, batch: Dict[str, Union[List, torch.LongTensor]], loss_config: DictConfig,
train=True):
    """Compute the SFT or DPO loss and other metrics for the given batch of inputs."""

    if loss_config.name in {'dpo', 'ipo'}:
        # policy模型针对chosen和rejected进行预测
        policy_chosen_logps, policy_rejected_logps = self.concatenated_forward(self.policy, batch)
        with torch.no_grad():
            # reference模型针对chosen和rejected进行预测
            reference_chosen_logps, reference_rejected_logps =
self.concatenated_forward(self.reference_model, batch)

        if loss_config.name == 'dpo':
            loss_kwargs = {'beta': loss_config.beta, 'reference_free': loss_config.reference_free,
'label_smoothing': loss_config.label_smoothing, 'ipo': False}
        elif loss_config.name == 'ipo':
            loss_kwargs = {'beta': loss_config.beta, 'ipo': True}
        else:
            raise ValueError(f'unknown loss {loss_config.name}')

        # 损失计算
        losses, chosen_rewards, rejected_rewards = preference_loss(
            policy_chosen_logps, policy_rejected_logps, reference_chosen_logps,
            reference_rejected_logps, **loss_kwargs)

        reward_accuracies = (chosen_rewards > rejected_rewards).float()

    elif loss_config.name == 'sft':
        policy_chosen_logits = self.policy(batch['chosen_input_ids'],
attention_mask=batch['chosen_attention_mask']).logits.to(torch.float32)
        policy_chosen_logps = _get_batch_logps(policy_chosen_logits, batch['chosen_labels'],
average_log_prob=False)

        losses = -policy_chosen_logps

```

```
return losses.mean()
```

3. LM的交叉熵计算 代码实现?

```
def _get_batch_logps(logits: torch.FloatTensor, labels: torch.LongTensor, average_log_prob: bool =
False) -> torch.FloatTensor:
    # 经模型后的logits进行批量计算logps

    assert logits.shape[:-1] == labels.shape

    # 基于先前的token预测下一个token
    labels = labels[:, 1:].clone()
    logits = logits[:, :-1, :]
    loss_mask = (labels != -100)

    # dummy token; we'll ignore the losses on these tokens later
    labels[labels == -100] = 0

    # 交叉熵函数
    per_token_logps = torch.gather(logits.log_softmax(-1), dim=2,
index=labels.unsqueeze(2)).squeeze(2)

    if average_log_prob:
        return (per_token_logps * loss_mask).sum(-1) / loss_mask.sum(-1)
    else:
        return (per_token_logps * loss_mask).sum(-1)
```